Adaptive Neural Networks

Neurons as Trainable Neural Networks

Joachim De Witte

May 2026

Abstract

Classical artificial neurons implement a fixed computational template: a linear transformation followed by a hand–crafted activation function. While effective, this design severely limits the internal complexity of neurons and forces expressivity to arise almost exclusively from network depth and width. In contrast, biological neurons exhibit rich internal structure, nonlinear dendritic processing, and context–dependent sensitivity. This paper introduces a theoretical framework for *adaptive neurons*, in which each activation function is replaced by a small trainable neural network. Every layer possesses its own activation network $f(x)$ and a corresponding derivative network $f'(x)$, enabling neurons to learn both their transformation and their local sensitivity. These internal networks provide persistent computational structure, support recursive "network–within–a–neuron" dynamics, and allow expressivity to grow without architectural expansion.

We formalize the adaptive neuron as a differentiable module with learnable internal geometry, derive its training dynamics, and introduce a consistency constraint linking f and f'. We further show that the framework naturally supports a staged optimization protocol: activation networks can be pre-trained to approximate classical nonlinearities, jointly refined with network weights, and finally frozen for a dedicated fine-tuning phase. This separation between learning activation geometry and learning task-specific representations introduces a new axis of inductive bias and stability control.

Within the Triadic Cosmos framework, adaptive neurons decompose into energy (transformation), information (parameters), geometry (internal structure), and recursion (self–application). The result is a new class of computational units that extend the representational capacity of multilayer perceptrons while remaining fully compatible with standard optimization methods. This paper provides the complete theoretical foundation required for practical implementation and future empirical exploration.

Contents

1 Introduction

Artificial neural networks are built upon a remarkably simple computational primitive: a neuron that computes a linear transformation followed by a fixed, hand–crafted activation function. This design has remained essentially unchanged since the earliest multilayer perceptrons. Although modern architectures have grown deeper, wider, and more structured, the internal mechanism of the neuron itself has remained static. As a result, the expressive power of neural networks is forced to emerge almost entirely from architectural scale rather than from the computational richness of individual units.

In contrast, biological neurons exhibit substantial internal complexity. Dendritic subunits perform nonlinear transformations, local integration, and context–dependent modulation. The neuron is not a pointwise nonlinearity but a structured dynamical system with internal geometry and adaptive sensitivity. This discrepancy between biological and artificial neurons raises a fundamental question: can artificial neurons be endowed with internal structure while remaining compatible with standard differentiable learning?

This paper develops a theoretical framework for *adaptive neurons*, a new class of computational units in which the activation function is replaced by a small trainable neural network. Each layer possesses its own activation network $f(x)$ and a corresponding derivative network $f'(x)$, enabling neurons to learn both their transformation and their local sensitivity. These internal networks introduce persistent computational structure, support recursive "network–within–a–neuron" dynamics, and allow representational capacity to grow without increasing the depth or width of the surrounding architecture.

The proposed construction generalizes learnable activation functions, subsumes hypernetwork–based modulation, and provides a differentiable analogue of dendritic computation. Unlike previous approaches, adaptive neurons explicitly model both the activation and its derivative, enabling stable integration into gradient–based optimization. The framework also admits a natural interpretation within the Triadic Cosmos paradigm: the activation network represents energy (transformation), its parameters encode information, its architecture defines geometry, and the recursive application of internal networks expresses self–referential computation.

Beyond the structure of the neuron itself, the framework naturally supports staged optimization procedures. Because activation networks are trainable modules, they may first be pre-trained to approximate classical nonlinearities, then jointly refined with network weights on the target task, and finally frozen for a dedicated fine-tuning phase. This separation between learning activation geometry and learning task-specific representations introduces a new methodological axis not present in classical architectures and suggests potential benefits for stability, inductive bias, and generalization.

The goal of this paper is purely theoretical. We provide formal definitions, derive the training dynamics, analyze the structural properties of adaptive neurons, and articulate their conceptual implications. No empirical claims are made. Instead, we establish the mathematical and architectural foundations required for future practical implementations and experimental exploration.

2 Related Work

The classical artificial neuron, defined as a linear transformation followed by a fixed activation function, has remained largely unchanged since the early development of multilayer perceptrons. Several research directions have attempted to increase the expressivity of neurons by modifying or extending the activation mechanism. These approaches provide important conceptual precursors to the present work. Below, we review the most relevant lines of research, highlight their advantages, and identify the limitations that motivate the introduction of adaptive neurons with learnable internal dynamics.

2.1 Learnable Activation Functions

A first family of approaches replaces fixed nonlinearities with parameterized or learnable activation functions. Examples include PReLU He et al. 2015, Adaptive Piecewise Linear Units (APL) Agostinelli et al. 2015, learnable spline activations, and rational activation functions Boulle, Townsend, and Webb 2020. These methods demonstrated that even small degrees of flexibility in the activation function can improve optimization dynamics, representation power, and empirical performance.

Advantages. Learnable activations showed that nonlinearities need not be static, that data–dependent shaping of activation curves can be beneficial, and that expressivity can be increased without adding layers or widening networks.

Limitations. Despite their benefits, these activations remain low–dimensional objects: they modify a curve, not a computational mechanism. They lack internal structure, cannot represent complex transformations, and do not provide an explicit model of the derivative $f'(x)$. As a result, they cannot capture the kind of internal dynamics characteristic of biological neurons.

2.2 Hypernetworks and Context–Dependent Activations

A second line of work uses hypernetworks to generate activation parameters dynamically Ha, Dai, and Le 2017. In these models, a small auxiliary network outputs the parameters of an activation function conditioned on the input, layer, or task context. This introduces a form of meta–adaptation and demonstrates that activations can themselves be outputs of learned computation.

Advantages. Hypernetwork–based activations show that nonlinearities can be context–sensitive and that small networks can modulate the behavior of larger ones. They provide a conceptual bridge between fixed activations and fully learned internal mechanisms.

Limitations. The activation is generated externally rather than implemented internally. The neuron itself remains structurally simple. Moreover, these methods do not model the derivative explicitly, and the generated activations often lack stability or interpretability. They do not provide a persistent internal computational structure within each neuron.

2.3 Dendritic Models in Computational Neuroscience

Biological neurons exhibit rich internal computation: dendritic subunits perform nonlinear transformations, integrate signals locally, and contribute to the overall firing behavior. Multi–compartment neuron models and dendritic processing frameworks Poirazi and Mel 2001 capture aspects of this complexity.

Advantages. These models provide strong evidence that internal structure within a neuron increases computational capacity. They motivate the idea that neurons are not point–like units but systems with internal geometry and dynamics.

Limitations. Biophysical models are difficult to integrate into differentiable machine learning frameworks. They typically rely on continuous–time dynamics, ion–channel models, or non–differentiable mechanisms. As such, they offer conceptual inspiration but not a practical computational analogue for deep learning.

2.4 Implicit Layers and Deep Equilibrium Models

Implicit layers and deep equilibrium models Bai, Kolter, and Koltun 2019 introduce internal iterative dynamics within a layer, showing that fixed–point computation can replace explicit depth. These models demonstrate that internal computation can increase expressivity without adding parameters linearly with depth.

Advantages. They highlight the value of internal computation and recursive structure within a layer.

Limitations. These models operate at the layer level, not the neuron level, and do not provide internal structure for individual neurons. They also lack explicit modeling of activation derivatives and are often challenging to train.

2.5 Staged Optimization and Activation Pre-Training

A smaller body of work has explored staged or progressive training procedures, such as layerwise pre-training Hinton, Osindero, and Teh 2006, curriculum learning, or the warm-starting of activation parameters. Some meta-learning approaches also adapt activation functions across tasks, and a few recent studies investigate pre-training of activation parameters to improve stability. These methods suggest that separating different phases of learning can improve optimization or generalization.

Advantages. Staged optimization demonstrates that neural networks may benefit from learning different components at different times. Pre-training activations can stabilize early training, while freezing certain parameters can reduce overfitting in later stages.

Limitations. Existing staged methods operate at the level of layers, architectures, or scalar activation parameters. None provide a principled framework for pre-training, jointly training, and freezing *internal activation networks* with their own derivative models. Moreover, prior work does not treat the activation mechanism as a computational module with persistent internal structure. As a result, existing staged approaches cannot capture the recursive "network-within-a-neuron" dynamics introduced in this paper.

2.6 Positioning of the Present Work

The approaches above collectively demonstrate three key insights: (i) nonlinearities benefit from being learnable, (ii) internal computation increases representational capacity, and (iii) biological neurons motivate richer internal structure. However, none of the existing methods provide a neuron whose activation function is itself a neural network, an explicit learnable model of the derivative $f'(x)$, persistent internal structure shared across neurons within a layer, or a recursive "network–within–a–neuron" architecture.

The adaptive neuron model proposed in this paper integrates these elements into a single construct. It generalizes learnable activations into full internal networks, introduces a learnable derivative network, and provides a triadic interpretation linking energy (transformation), information (parameters), geometry (internal structure), and recursion (self–application). This positions adaptive neurons as a new class of computational units that extend the expressive power of multilayer perceptrons without requiring architectural expansion.

3 Adaptive Neuron Architecture

Adaptive neurons replace the classical activation function with a pair of trainable neural networks: an activation network $f(x)$ and a derivative network $f'(x)$. This section formalizes their structure, forward computation, and internal geometry. The goal is to define a computational unit whose nonlinearity is no longer fixed but learned, while remaining fully compatible with gradient-based optimization.

3.1 Classical Neuron Structure

A standard artificial neuron computes

$$y = \phi(Wx + b),$$

where W and b are trainable parameters and ϕ is a fixed activation function such as tanh, ReLU, or GELU. The activation is pointwise, memoryless, and identical across all neurons in a layer. As a result, the expressive power of the network arises almost entirely from the arrangement of linear transformations.

3.2 Activation Networks

In an adaptive neuron, the activation function is replaced by a small multilayer perceptron:

$$f : \mathbb{R} \to \mathbb{R},$$

shared across all neurons in a layer. The forward computation becomes

$$y = f(Wx + b).$$

The activation network introduces internal structure, depth, and learnable geometry. Its parameters are persistent across training and define the nonlinear transformation performed by the layer.

A typical activation network may be defined as

$$f(x) = A_2\,\sigma(A_1 x + c_1) + c_2,$$

where σ is a classical nonlinearity (e.g., tanh), and A_1, A_2, c_1, c_2 are trainable parameters. The architecture may vary across layers, enabling heterogeneous activation geometries.

3.3 Derivative Networks

Backpropagation requires the derivative $f'(x)$. Instead of computing it analytically, adaptive neurons introduce a second neural network

$$f' : \mathbb{R} \to \mathbb{R},$$

trained to approximate the derivative of f. During training, the gradient of the loss with respect to the pre-activation $z = Wx + b$ is computed as

$$\frac{\partial L}{\partial z} = f'(z) \odot \frac{\partial L}{\partial y}.$$

The derivative network decouples the forward and backward computational pathways, allowing the backward sensitivity to be learned rather than imposed. This provides additional flexibility and may stabilize gradient flow.

3.4 Schematic Overview

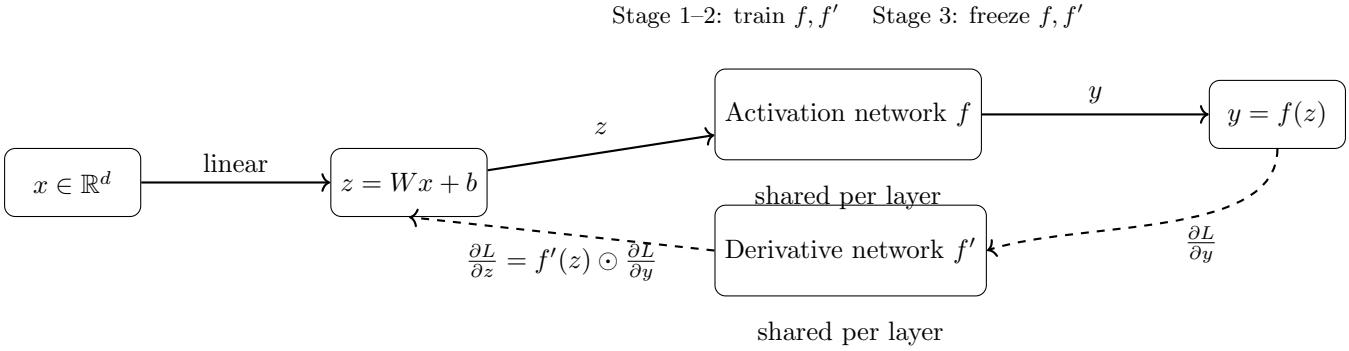

Figure 1: Schematic view of an adaptive neuron layer. The classical pre-activation $z = Wx+b$ is passed through a trainable activation network f, while a separate derivative network f' provides the backward sensitivity during training. Both f and f' are shared across neurons in the layer and can be trained in a three-stage protocol (pre-training, joint training, freezing).

3.5 Consistency Constraint

To ensure that $f'(x)$ remains a valid approximation of the derivative of $f(x)$, we introduce a consistency loss:

$$\mathcal{L}_{\text{consistency}} = \mathbb{E}_{x \sim \mathcal{D}} \left[\left\| f'(x) - \frac{d}{dx} f(x) \right\|^2 \right].$$

This term encourages coherence between the forward and backward pathways without requiring exact equality. The strength of this constraint can be tuned to balance flexibility and stability.

3.6 Internal Geometry and Recursive Structure

Because both f and f' are neural networks, each adaptive neuron contains a recursive computational structure: a network inside a neuron inside a network. This recursion is shallow but persistent, giving each layer its own internal geometry. The architecture of the activation network—its depth, width, and nonlinearities—defines the shape of the transformation performed by the layer.

3.7 Compatibility with Standard Architectures

Adaptive neurons can replace classical neurons in any multilayer perceptron without modifying the surrounding architecture. The only change is the substitution of the fixed activation function with the learned activation network. The derivative network is used only during training and may be discarded or simplified during inference.

This modularity ensures that adaptive neurons remain compatible with standard optimization methods, initialization schemes, and architectural patterns.

4 Training Dynamics

Adaptive neurons introduce trainable internal structure into the activation mechanism, which requires a careful formulation of the forward pass, backward pass, and optimization procedure.

This section formalizes the training dynamics of adaptive neurons and introduces a staged optimization protocol that separates the learning of activation geometry from the learning of task-specific representations.

4.1 Forward Computation

For an input vector $x \in \mathbb{R}^d$, a layer computes the pre-activation

$$z = Wx + b,$$

where W and b are the trainable parameters of the layer. The activation network f then produces the output

$$y = f(z).$$

Because f is itself a neural network, the forward computation of the layer includes both the linear transformation and the internal nonlinear structure of the activation network. This introduces persistent computational depth inside each neuron.

4.2 Backward Computation

Backpropagation requires the derivative of the activation with respect to its input. Instead of computing this derivative analytically, adaptive neurons use a learned derivative network $f'(x)$. Given the gradient of the loss with respect to the layer output,

$$\frac{\partial L}{\partial y},$$

the gradient with respect to the pre-activation is

$$\frac{\partial L}{\partial z} = f'(z) \odot \frac{\partial L}{\partial y}.$$

The gradients with respect to the layer parameters follow the classical form:

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial z}\right) x^\top, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}.$$

The activation network f and derivative network f' are trained by backpropagating through their internal layers in the usual way.

4.3 Consistency Loss

To ensure that the derivative network remains aligned with the activation network, we introduce a consistency term:

$$\mathcal{L}_{\text{consistency}} = \mathbb{E}_{z \sim \mathcal{D}} \left[\left\| f'(z) - \frac{d}{dz} f(z) \right\|^2 \right].$$

This term does not enforce exact equality—adaptive neurons are not required to satisfy analytic differentiability—but it encourages coherence between the forward and backward pathways. The strength of this constraint can be tuned to balance flexibility and stability.

4.4 Total Training Objective

For a supervised learning task with loss $\mathcal{L}_{\text{task}}$, the full objective becomes

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{cons}} \mathcal{L}_{\text{consistency}},$$

where λ_{cons} controls the influence of the consistency constraint.

Additional regularization terms may be applied to the activation networks, such as weight decay, Lipschitz penalties, or smoothness constraints, depending on the desired activation geometry.

4.5 Staged Optimization Protocol

Because activation networks are trainable modules, adaptive neurons naturally support a staged training procedure. We propose a three-stage protocol:

Stage 1: Pre-Training of Activation Networks. Activation networks f and f' are trained to approximate a classical activation function (e.g., tanh) and its derivative. This produces a stable initialization and avoids chaotic early dynamics.

Stage 2: Joint Training. The full network is trained end-to-end on the target task. Both the layer parameters (W, b) and the activation networks (f, f') are updated. During this phase, the activation geometry adapts to the structure of the data.

Stage 3: Freezing and Fine-Tuning. The activation networks are frozen, and only the layer parameters are further optimized. This reduces the degrees of freedom in late training, stabilizes optimization, and may improve generalization.

This staged protocol separates the learning of activation geometry from the learning of task-specific representations, providing a structured optimization pathway not available in classical architectures.

4.6 Computational Considerations

The presence of activation networks increases the computational cost of both forward and backward passes. However, because the activation networks are shared across all neurons in a layer, the overhead grows with the number of layers rather than the number of units. Furthermore, the derivative network is only required during training and may be discarded or simplified during inference.

4.7 Compatibility with Standard Optimization

Despite their internal complexity, adaptive neurons remain fully compatible with standard optimization methods such as SGD, Adam, RMSProp, and second-order approximations. No modifications to the surrounding architecture are required. The only difference is the replacement of the fixed activation function with a learned activation network and its derivative.

5 Method

This section formalizes the computational structure of adaptive neurons, derives their forward and backward dynamics, and introduces a staged optimization protocol that separates the learning of activation geometry from the learning of task-specific representations. The goal is to provide a complete and self-contained mathematical description of the model.

5.1 Pre-Activation and Layer Structure

Consider a layer with input $x \in \mathbb{R}^d$. The classical pre-activation is preserved:

$$z = Wx + b,$$

where $W \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$ are trainable parameters. The novelty lies entirely in the activation mechanism applied to z.

5.2 Activation Network

Each layer possesses a trainable activation network

$$f : \mathbb{R} \to \mathbb{R},$$

shared across all neurons in the layer. A typical architecture is a small multilayer perceptron:

$$f(x) = A_2\, \sigma(A_1 x + c_1) + c_2,$$

where σ is a classical nonlinearity (e.g., tanh), and (A_1, A_2, c_1, c_2) are trainable parameters. The layer output is

$$y = f(z).$$

This introduces persistent internal structure and recursive computation inside each neuron.

5.3 Derivative Network

Backpropagation requires the derivative of the activation. Instead of computing it analytically, adaptive neurons introduce a second network:

$$f' : \mathbb{R} \to \mathbb{R}.$$

During training, the gradient with respect to the pre-activation is computed as

$$\frac{\partial L}{\partial z} = f'(z) \odot \frac{\partial L}{\partial y}.$$

The derivative network is trained jointly with the activation network and the layer parameters.

5.4 Consistency Constraint

To ensure coherence between the forward and backward pathways, we introduce a consistency loss:

$$\mathcal{L}_{\text{consistency}} = \mathbb{E}_{z \sim \mathcal{D}} \left[\left\| f'(z) - \frac{d}{dz} f(z) \right\|^2 \right].$$

This term does not enforce exact equality—adaptive neurons are not required to be analytically differentiable—but it encourages the derivative network to approximate the true derivative of the activation network.

5.5 Total Loss Function

For a supervised learning task with loss $\mathcal{L}_{\text{task}}$, the full objective is

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{cons}} \mathcal{L}_{\text{consistency}},$$

where λ_{cons} controls the strength of the consistency constraint.

Additional regularization terms may be applied to the activation networks, such as smoothness penalties or Lipschitz constraints.

5.6 Backpropagation Through Adaptive Neurons

The backward pass proceeds as follows:

$$\frac{\partial L}{\partial y} \quad \text{(from next layer)},$$

$$\frac{\partial L}{\partial z} = f'(z) \odot \frac{\partial L}{\partial y},$$

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial z}\right) x^\top, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}.$$

Gradients propagate through the internal layers of f and f' using standard automatic differentiation.

5.7 Staged Optimization Protocol

Adaptive neurons naturally support a three-stage training procedure:

Stage 1: Pre-Training of Activation Networks. Activation networks f and f' are trained to approximate a classical activation function (e.g., tanh) and its derivative. This produces a stable initialization and avoids chaotic early dynamics.

Stage 2: Joint Training. The full network is trained end-to-end on the target task. Both the layer parameters (W, b) and the activation networks (f, f') are updated. During this phase, the activation geometry adapts to the structure of the data.

Stage 3: Freezing and Fine-Tuning. The activation networks are frozen, and only the layer parameters are further optimized. This reduces the degrees of freedom in late training, stabilizes optimization, and may improve generalization.

This staged protocol separates the learning of activation geometry from the learning of task-specific representations.

5.8 Computational Considerations

The activation and derivative networks introduce additional computation during training. However:

- the overhead is per-layer, not per-neuron,
- the derivative network is only required during training,
- activation networks can be cached or compiled for efficient inference.

Thus, the computational cost remains manageable, especially for small activation networks.

5.9 Compatibility with Standard Architectures

Adaptive neurons can replace classical neurons in any multilayer perceptron without modifying the surrounding architecture. They remain compatible with:

- SGD, Adam, RMSProp, and second-order optimizers,
- standard initialization schemes,
- batching, normalization, and regularization techniques.

The only change is the substitution of the fixed activation function with a learned activation network and its derivative.

6 Theoretical Properties

Adaptive neurons introduce internal computational structure into the activation mechanism, fundamentally altering the expressive and geometric properties of multilayer perceptrons. This section analyzes the theoretical implications of this design, focusing on expressivity, approximation power, parameter scaling, internal geometry, and stability.

6.1 Expressive Power

Classical multilayer perceptrons rely on depth and width to increase expressive capacity. In contrast, adaptive neurons introduce additional depth inside each activation function. For a layer with activation network f, the effective computation becomes

$$x \mapsto f(Wx + b),$$

where f itself may contain multiple nonlinear transformations. This yields a composite mapping with strictly greater expressive power than a classical layer of the same width.

Formally, if f is a universal approximator on compact subsets of $\mathbb{R}$, then the adaptive neuron layer is a universal approximator on compact subsets of $\mathbb{R}^d$ even when the surrounding architecture is shallow. This suggests that adaptive neurons can approximate functions that would otherwise require significantly deeper or wider networks.

6.2 Approximation Power and Functional Complexity

The activation network f allows each layer to implement a nonlinear transformation whose complexity is independent of the number of neurons in the layer. This decouples functional complexity from architectural scale. In particular:

- A single adaptive layer can approximate multi-regime nonlinearities that classical activations cannot represent.
- The derivative network $f'(x)$ enables the backward sensitivity to adapt to the structure of the task, potentially improving optimization.
- The internal geometry of f allows the layer to implement transformations that resemble multi-branch or piecewise-smooth functions.

These properties suggest that adaptive neurons may reduce the need for architectural depth in certain regimes.

6.3 Parameter Scaling

Let P denote the number of parameters in the linear transformation of a layer and Q the number of parameters in the activation network. The total parameter count becomes

$$P_{\text{total}} = P + Q.$$

Because Q is shared across all neurons in the layer, the relative overhead decreases as the width increases. This yields a favorable scaling regime:

$$\frac{Q}{P} \to 0 \quad \text{as width} \to \infty.$$

Thus, adaptive neurons introduce a fixed per-layer cost while potentially replacing the need for additional layers or wider architectures. This trade-off between internal complexity and architectural scale is a central theoretical implication of the model.

6.4 Internal Geometry and Recursive Structure

The activation network introduces a recursive computational structure: a network inside a neuron inside a network. This recursion is shallow but persistent, giving each layer its own internal geometry. The architecture of the activation network—its depth, width, and nonlinearities—defines the shape of the transformation performed by the layer.

This internal geometry enables:

- hierarchical nonlinear transformations within a single layer,
- context-dependent sensitivity through the learned derivative network,
- layer-specific activation regimes that differ across the network.

These properties mirror the diversity of nonlinear processing observed in biological neurons.

6.5 Stability and Gradient Flow

The derivative network $f'(x)$ introduces a learned backward pathway. This may improve gradient flow by adapting the sensitivity of the layer to the structure of the task. However, it also introduces potential instability:

- If $f'(x)$ becomes too large, gradients may explode.
- If $f'(x)$ becomes too small, gradients may vanish.
- If f and f' diverge, optimization may become inconsistent.

The consistency constraint mitigates these issues by encouraging coherence between the forward and backward pathways. The staged training protocol further stabilizes optimization by freezing activation networks in late training.

6.6 Inductive Bias and Functional Specialization

Because each layer possesses its own activation network, adaptive neurons introduce a new axis of inductive bias: the geometry of the activation itself. This enables:

- early layers to learn smooth, compressive nonlinearities,
- deeper layers to learn sharper or more selective transformations,

- task-specific specialization of activation geometry.

This specialization resembles the hierarchical organization of biological cortical layers and may improve representation learning.

6.7 Compatibility with Classical Theory

Despite their internal complexity, adaptive neurons remain compatible with classical results on:

- universal approximation,
- differentiability and gradient-based optimization,
- Lipschitz continuity (with appropriate regularization),
- compositional function approximation.

The framework extends classical theory without violating its assumptions, providing a principled generalization of the standard neuron model.

7 Testable Predictions

Although this paper is purely theoretical, the adaptive neuron framework yields several concrete predictions that can be empirically evaluated in future work. These predictions concern expressivity, optimization behavior, data efficiency, architectural scaling, and the functional role of the learned derivative network. Each prediction is stated in a falsifiable form, enabling direct experimental verification.

7.1 Prediction 1: Increased Expressive Power per Parameter

Adaptive neurons introduce internal activation networks that are shared across all units in a layer. This suggests that a network with significantly fewer global parameters may approximate functions typically requiring much larger architectures. Formally, we predict that for a fixed parameter budget, models with adaptive neurons will approximate a broader class of nonlinear functions than classical multilayer perceptrons. This prediction can be tested by comparing approximation error on synthetic regression tasks under matched parameter counts.

7.2 Prediction 2: Faster or More Stable Learning Dynamics

Because the derivative network $f'(x)$ is learned rather than analytically derived, it may adapt to improve gradient flow or reduce pathological curvature in the loss landscape. We predict that, under identical initialization and optimization settings, adaptive neurons will exhibit either faster convergence or reduced variance in training dynamics compared to classical activations. This can be evaluated by measuring convergence rates and gradient statistics across multiple random seeds.

7.3 Prediction 3: Improved Data Efficiency

If the internal activation networks capture task-specific nonlinear structure, the model may require fewer training samples to achieve comparable performance. We predict that adaptive neurons will outperform classical networks in low-data regimes, particularly on tasks with complex or highly non-monotonic nonlinearities. This can be tested by training both models on progressively smaller subsets of a dataset.

7.4 Prediction 4: Layer-Specific Activation Geometry Yields Functional Specialization

Since each layer possesses its own activation network, we predict that the learned activation functions will diverge across layers and acquire distinct computational roles. Early layers may learn smoother or more compressive nonlinearities, while deeper layers may learn sharper or more selective transformations. This can be tested by visualizing the learned activation functions and their derivatives after training.

7.5 Prediction 5: Pre-Training on Classical Activations Improves Stability

The framework predicts that initializing f and f' to approximate a classical activation (e.g., tanh) will yield more stable early training dynamics than random initialization. This can be tested by comparing training curves, gradient norms, and activation statistics under both initialization schemes.

7.6 Prediction 6: The Derivative Network Is Primarily Useful During Training

Because $f'(x)$ is only required for backpropagation, we predict that the derivative network may be discarded or simplified after training with minimal impact on inference performance. This can be tested by replacing $f'(x)$ with an analytic derivative of $f(x)$, a numerical approximation, or a frozen version of the learned derivative network during inference.

7.7 Prediction 7: Transformer-Like Inductive Biases Emerge at the Neuron Level

The internal multi-layer structure of the activation networks resembles the feedforward blocks of transformer architectures. We predict that adaptive neurons will exhibit transformer-like behaviors, such as hierarchical feature formation or improved modularity, even in simple multilayer perceptrons. This can be tested by analyzing representation similarity, feature clustering, or layer-wise abstraction patterns.

7.8 Prediction 8: Consistency Constraints Improve Generalization

The requirement that $f'(x)$ approximate the derivative of $f(x)$ acts as a structural regularizer. We predict that models trained with this consistency constraint will generalize better than those trained without it, even when the forward performance during training is similar. This can be tested by comparing validation performance and robustness to distribution shifts.

7.9 Prediction 9: Three-Stage Training Improves Final Performance

The proposed three-stage training protocol—(i) pre-training activation networks on a fixed nonlinearity, (ii) joint training of network weights and neuron networks, and (iii) freezing neuron networks for a final fine-tuning phase—introduces a structured learning schedule. We predict that this staged approach will yield better final performance than purely joint training, due to improved stability in early training and reduced overfitting in late training. This can be tested by comparing final loss and generalization error across training regimes.

7.10 Prediction 10: Three-Stage Training Reduces Sensitivity to Initialization

Because the first stage produces a canonical activation geometry and the third stage removes degrees of freedom by freezing neuron networks, we predict that the three-stage protocol will

reduce sensitivity to random initialization. Models trained with this protocol should exhibit lower variance in performance across random seeds compared to models trained jointly from scratch. This can be evaluated by measuring performance dispersion across multiple independent training runs.

In summary, the adaptive neuron framework yields a set of clear, falsifiable predictions that can be evaluated using controlled toy experiments. These predictions provide a roadmap for future empirical work and offer a means of assessing the practical relevance of the theoretical contributions presented in this paper.

8 Novelties and Implications

The adaptive neuron framework introduces several conceptual and structural innovations that distinguish it from classical neural network design. These novelties have direct implications for expressivity, training dynamics, architectural modularity, and the theoretical understanding of neural computation.

8.1 Neural Networks as Activation Functions

The central novelty of this work is the replacement of fixed activation functions with trainable neural networks. Each layer is equipped with its own activation network $f(x)$, enabling the nonlinearity itself to acquire internal structure, depth, and learnable geometry. This transforms the activation from a static mathematical function into a computational module with representational capacity. As a result, expressivity no longer depends solely on the depth or width of the surrounding architecture but also on the internal dynamics of the activation networks.

8.2 A Separate Network for the Derivative

A second innovation is the introduction of a dedicated derivative network $f'(x)$. Instead of relying on analytic differentiation of f, the model learns an explicit approximation of the derivative. This design decouples the forward and backward computational pathways, allowing the derivative to be optimized for stability, smoothness, or task–specific sensitivity. Although $f'(x)$ is essential during training, it may be discarded or simplified after optimization, raising the possibility of reduced inference cost.

8.3 Layer-Specific Activation Geometry

Unlike classical networks, where the same activation function is applied uniformly across layers, adaptive neurons allow each layer to possess its own activation geometry. The internal architecture of the activation network—its depth, width, and nonlinearities—can differ across layers, enabling heterogeneous computational roles. This mirrors the diversity of nonlinear processing observed in biological cortical layers and introduces a new dimension of architectural design: the geometry of the activation itself.

8.4 Initialization and Pre-Training of Activation Networks

Because activation networks are themselves trainable modules, their initialization is nontrivial. A key implication of this framework is the need for principled pre-training. Initializing f and f' to approximate a classical activation function (e.g., tanh) provides a stable starting point and avoids chaotic early dynamics. This pre-training step introduces a new methodological axis: the choice of initial activation geometry and its influence on subsequent learning.

8.5 A Three-Stage Training Protocol

A further novelty arises from the staged training dynamics enabled by adaptive neurons. The framework naturally supports a three-stage protocol: (i) pre-training activation networks to approximate a fixed nonlinearity, (ii) joint training of network weights and neuron networks on the target task, and (iii) freezing the neuron networks for a final fine-tuning phase. This staged approach separates the learning of activation geometry from the learning of task-specific representations, providing a structured optimization pathway not available in classical architectures. It also introduces the possibility of reusing pre-trained neuron networks across tasks, analogous to transfer learning at the activation level.

8.6 Parameter Scaling and Effective Capacity

Adaptive neurons alter the relationship between parameter count and expressive power. For a layer with P parameters in its weight matrix and Q parameters in its activation network, the effective parameterization becomes approximately $P + Q$. However, because the activation network is shared across all neurons in the layer, the increase in parameters is modest relative to the potential gain in expressivity. This raises the possibility that a network with significantly fewer global parameters may approximate functions typically requiring much larger architectures. Understanding this trade-off is an important theoretical implication of the model.

8.7 Potential for Improved Learning Dynamics

The presence of a learnable derivative network introduces the possibility of improved gradient flow, smoother optimization landscapes, and task-adaptive sensitivity. The three-stage training protocol further suggests that freezing activation networks in the final phase may reduce overfitting and stabilize late-stage optimization. Although this paper does not make empirical claims, the theoretical structure indicates that adaptive neurons may learn more efficiently or require fewer data points in certain regimes.

8.8 Connections to Transformer-Like Computation

The internal multi-layer structure of activation networks resembles, in miniature, the feedforward blocks of transformer architectures. This structural similarity raises the possibility that adaptive neurons inherit some of the inductive biases that make transformers effective, such as hierarchical representation learning or modular nonlinear processing. If so, adaptive neurons may serve as a bridge between classical multilayer perceptrons and transformer-style computation, offering a compact and theoretically grounded alternative.

In summary, the adaptive neuron framework introduces a new computational primitive that expands the design space of neural architectures. By endowing neurons with internal networks for both activation and derivative computation, and by enabling staged optimization protocols, it opens new avenues for expressivity, modularity, and theoretical analysis, while remaining fully compatible with standard gradient-based optimization.

9 Limitations and Open Questions

The adaptive neuron framework proposed in this paper introduces internal computational structure into individual neurons, but several limitations and open questions remain. These issues are not flaws of the model but natural consequences of extending the classical neuron design. They identify the boundaries of the present theory and outline directions for future investigation.

9.1 Computational Overhead

The most immediate limitation concerns computational cost. Each adaptive neuron relies on two internal networks, $f(x)$ and $f'(x)$, which must be evaluated during forward and backward passes. Although these internal networks are intentionally small, their presence introduces nontrivial overhead compared to fixed activations. An open question is whether the increased expressivity compensates for the additional computation in practical settings, and whether optimized implementations or architectural constraints can reduce this cost.

9.2 Parameter Efficiency

A central motivation for adaptive neurons is the possibility of achieving richer functional expressivity with fewer global parameters. However, it remains theoretically unresolved whether the internal networks of adaptive neurons can consistently replace depth or width in large architectures. Formal bounds on expressivity, sample complexity, or approximation power would clarify whether adaptive neurons can serve as a principled alternative to scaling network size.

9.3 Learning Dynamics and Data Efficiency

The introduction of internal networks raises questions about optimization behavior. It is unclear whether adaptive neurons enable faster convergence, improved conditioning of the loss landscape, or reduced data requirements. The presence of a learnable derivative network $f'(x)$ may stabilize gradient flow, but this remains a hypothesis. Understanding how adaptive neurons interact with gradient descent, initialization schemes, and regularization is an open theoretical challenge.

9.4 Interaction with Transformer-Like Structure

Adaptive neurons share structural features with transformer feedforward blocks: internal multilayer computation, learnable nonlinear transformations, and recursive composition. Whether these shared properties yield transformer-like benefits—such as improved representation learning, modularity, or hierarchical abstraction—remains unknown. A deeper theoretical analysis is required to determine whether adaptive neurons inherit any of the inductive biases that make transformers effective.

9.5 Stability and Regularization

The flexibility of internal activation networks introduces potential instability. Without appropriate constraints, $f(x)$ or $f'(x)$ may become excessively sharp, oscillatory, or poorly conditioned. The consistency requirement between f and f' provides partial control, but the optimal form and strength of such constraints remain open questions. Identifying principled regularization strategies is essential for ensuring stable training and well-behaved internal dynamics.

9.6 Scope of Applicability

It is not yet clear which classes of problems benefit most from adaptive neurons. Their internal structure may be advantageous for tasks requiring fine-grained nonlinear transformations, but unnecessary for simpler domains. Determining the regimes in which adaptive neurons provide meaningful advantages is an important direction for future theoretical and empirical work.

9.7 Complexity of Staged Training Protocols

The introduction of staged training protocols—such as pre-training activation networks, joint training, and final fine-tuning with frozen neurons—adds methodological complexity. It remains

unclear whether the benefits of staged training outweigh the additional hyperparameters, training phases, and implementation overhead. Understanding when staged training is beneficial, and whether simpler alternatives exist, is an open question.

9.8 Optimal Pre-Training Targets

The three-stage protocol relies on pre-training activation networks to approximate a fixed nonlinearity (e.g., tanh). However, it is unknown which pre-training targets yield the best downstream performance. Alternatives such as GELU, rational functions, mixtures of nonlinearities, or task-specific synthetic objectives may produce more effective activation geometries. Identifying optimal pre-training strategies is an open research direction.

9.9 Freezing Strategies and Timing

In the final stage of the three-stage protocol, neuron networks are frozen to stabilize optimization. However, the optimal timing of freezing, the degree of allowed fine-tuning, and the interaction between freezing and generalization remain unexplored. It is possible that premature freezing limits expressivity, while late freezing reduces stability. Systematic investigation is required to understand these trade-offs.

9.10 Reusability and Transferability of Pre-Trained Neurons

If activation networks can be pre-trained independently of downstream tasks, an open question is whether such neurons can be reused across architectures or domains. This raises the possibility of a library of pre-trained neuron types, analogous to transfer learning at the activation level. Whether such transfer is feasible, beneficial, or stable remains unknown.

In summary, adaptive neurons offer a promising new computational primitive, but their practical and theoretical properties are not yet fully understood. Addressing these limitations and open questions—including those introduced by staged training protocols—will be essential for assessing the broader impact of this framework on neural network design.

10 Conclusion

This paper introduced a theoretical framework for *adaptive neurons*: computational units whose activation functions are themselves trainable neural networks. By equipping each layer with an internal activation network $f(x)$ and a corresponding derivative network $f'(x)$, adaptive neurons acquire persistent internal structure, learnable sensitivity, and recursive computational depth. This design generalizes classical activation functions, extends the expressive capacity of multilayer perceptrons without architectural expansion, and provides a differentiable analogue of dendritic processing in biological neurons.

The framework developed here is intentionally and entirely theoretical. We formalized the structure of adaptive neurons, derived their training dynamics, and articulated the consistency constraints that couple f and f'. We also positioned the model within existing research on learnable activations, hypernetwork modulation, dendritic computation, and implicit layers, showing that adaptive neurons unify and extend these ideas while avoiding their limitations. Within the Triadic Cosmos paradigm, adaptive neurons naturally decompose into energy (transformation), information (parameters), geometry (internal architecture), and recursion (self–application), offering a principled conceptual grounding for their design.

Beyond the structure of individual neurons, the framework also enables new training methodologies. In particular, the proposed three-stage protocol—pre-training activation networks on a fixed nonlinearity, jointly training neurons and network weights on the target task, and finally

freezing the neuron networks for a dedicated fine-tuning phase—introduces a staged optimization pathway not available in classical architectures. This separation between learning activation geometry and learning task-specific representations suggests new forms of inductive bias, stability control, and potential reuse of pre-trained neuron networks across tasks.

No empirical claims were made, and no implementation details were required. Instead, the goal of this work was to establish a complete theoretical foundation that is sufficiently precise to enable practical realization without further conceptual development. The resulting framework opens several avenues for future research, including the study of neurons with temporal dynamics, internal memory, stochastic substructures, or deeper recursive organization, as well as the systematic evaluation of staged training protocols. Adaptive neurons thus represent a new direction in the design of computational primitives for neural networks, one in which expressivity arises not only from network architecture but from the internal dynamics and learnable geometry of the neurons themselves.

A Triadic Cosmos Ecosystem Context

This paper is part of the *Triadic Cosmos* ecosystem, which develops a unified conceptual and computational framework spanning physical ontology and modular architectures for intelligent systems. All materials, extended notes, and evolving documentation are available in the public repository: `https://github.com/triadic-cosmos`.

References

Agostinelli, Forest, Matthew Hoffman, Peter Sadowski, and Pierre Baldi (2015). "Learning Activation Functions to Improve Deep Neural Networks". In: *International Conference on Learning Representations (ICLR)*.

Bai, Shaojie, J. Zico Kolter, and Vladlen Koltun (2019). "Deep Equilibrium Models". In: *Advances in Neural Information Processing Systems (NeurIPS)*.

Boulle, Nicolas, Alex Townsend, and Matthew Webb (2020). "Rational Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*.

Ha, David, Andrew M. Dai, and Quoc V. Le (2017). "HyperNetworks". In: *International Conference on Learning Representations (ICLR)*.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

Hinton, Geoffrey E, Simon Osindero, and Yee-Whye Teh (2006). "A fast learning algorithm for deep belief nets". In: *Neural Computation* 18.7, pp. 1527–1554.

Poirazi, Panayiota and Bartlett W. Mel (2001). "Impact of Active Dendrites and Structural Plasticity on the Computational Properties of Neurons". In: *Neuron* 29.3, pp. 779–796.